

Chapter 5 (PART 2)

Structural Model

Component Diagram

ลักษณะของ Component Diagram

- 1) เป็น Structured Diagram ประเภท Static Diagram แสดงโครงสร้างทางกายภาพของ Software ซึ่งแสดงให้เห็นถึง **ส่วนประกอบของ Software (Software Component หรือเรียกว่า Components)** ต่างๆ ของระบบ รวมถึงความสัมพันธ์ระหว่าง Components ต่างๆ
- 2) Component Diagram จะมีความสัมพันธ์กับ **Class Diagram** เพราะในแต่ละ Component นั้น จะประกอบไปด้วย Class ที่มีอยู่ในระบบตั้งแต่ 1 Class ขึ้นไป
- 3) ประโยชน์ของ Component Diagram คือสามารถ **แบ่งระบบงาน (System) ขนาดใหญ่ออกเป็นระบบย่อยๆ (Subsystem)** ซึ่งแต่ละ Subsystem ก็จะมี Component ต่างๆ ประกอบอยู่ การแบ่งระบบออกเป็น Subsystem ทำให้การพัฒนาระบบมีประสิทธิภาพและสนับสนุนหลักการพัฒนาระบบงานแบบเป็นทีมงานที่สามารถแบ่งส่วนย่อยต่างๆ ให้รับผิดชอบงานในแต่ละส่วนงานย่อยได้

System Decomposition

-คือ กระบวนการในขั้นตอนออกแบบ S/W ของระบบ ซึ่ง**ระบบที่มีความซับซ้อน**มาก
นิยมออกแบบโดยแยกระบบออกเป็นส่วนย่อยๆ หรือ Subsystem

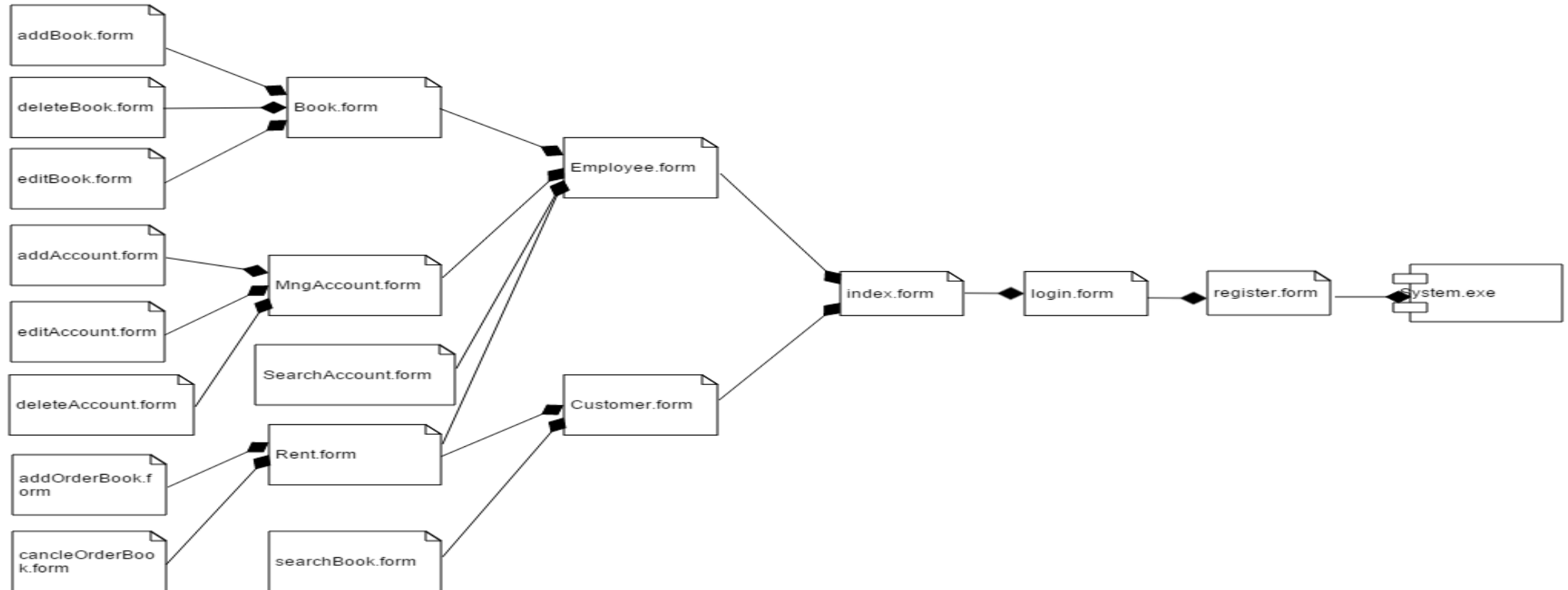
-หลักการแยกระบบ ออกเป็น **3 Subsystem** คือ

1) Presentation Logic Subsystem : ทำหน้าที่ติดต่อสื่อสารกับผู้ใช้งานโดยตรง เช่น
User Interface, Output unit, Input unit

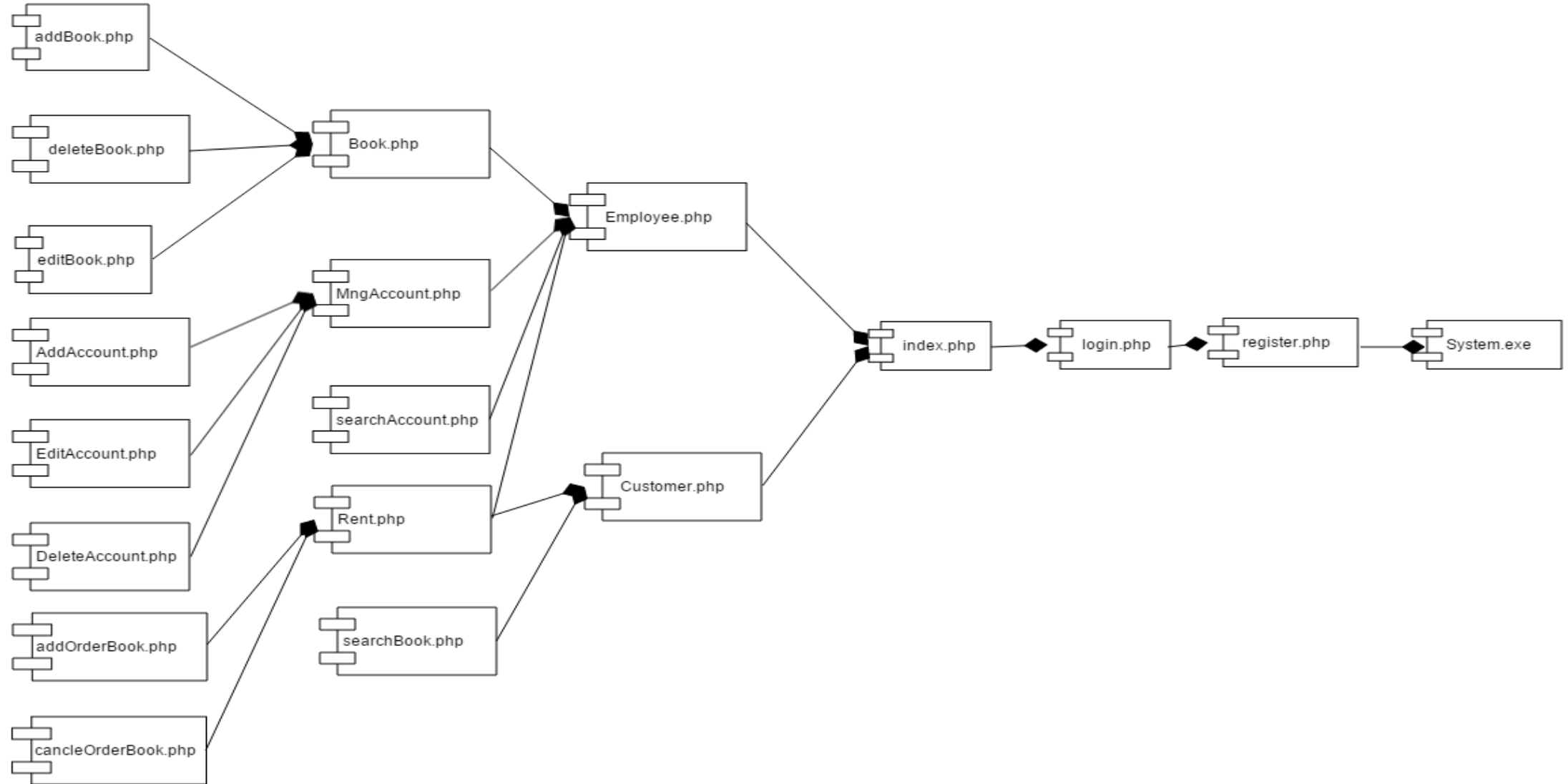
2) Working Logic Subsystem : เกี่ยวกับการทำงานที่เกิดขึ้นจริงใน CPU ของ
คอมพิวเตอร์ เช่น การบันทึกข้อมูลสั่งซื้อ, การปรับปรุงสถานะการชำระเงิน

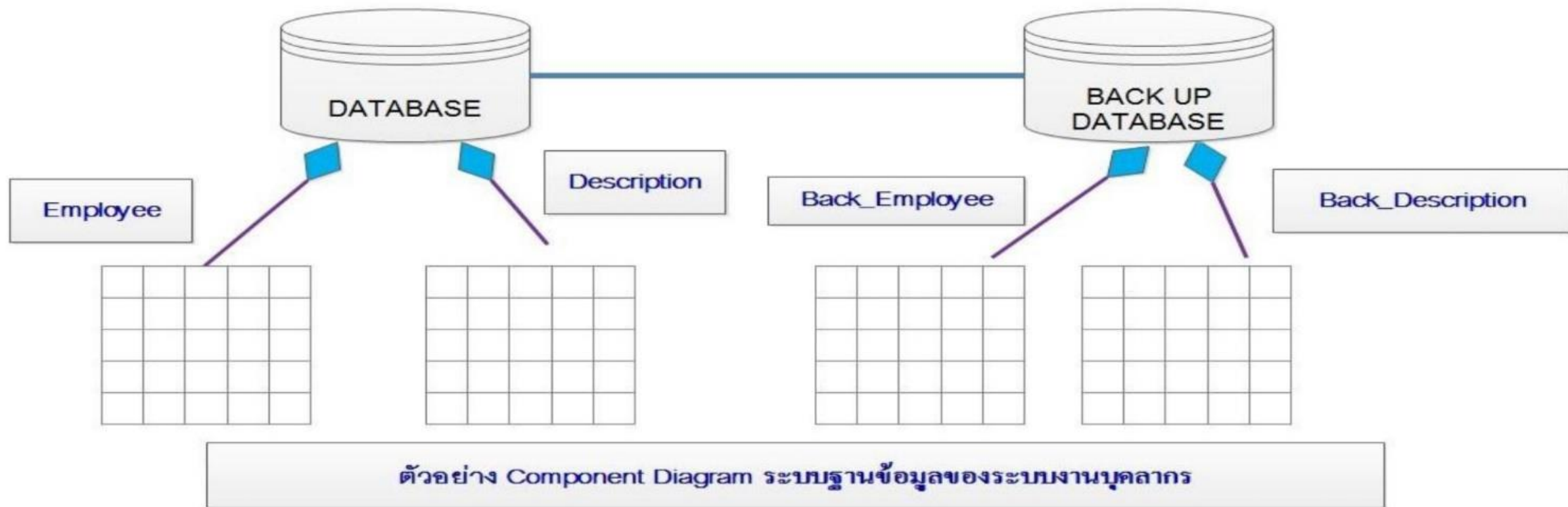
3) Database Logic Subsystem : เป็น Model ของ Data Item ต่างๆที่ถูกเก็บใน
Database หรือ File

ตัวอย่าง Presentation Logic Subsystem



ตัวอย่าง Working Logic Subsystem

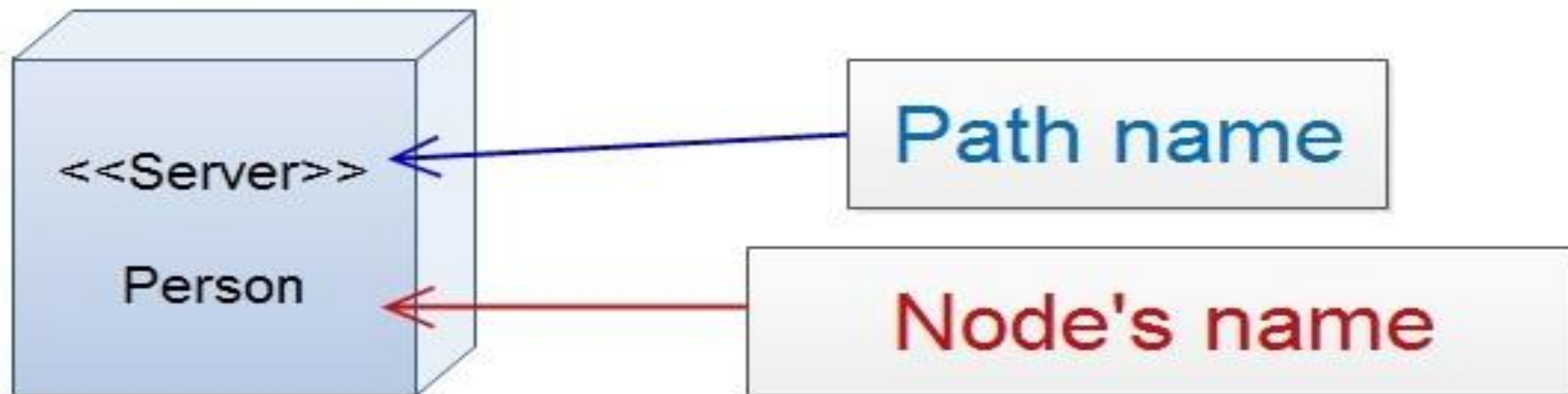




Ex. 2 : เป็น Component Diagram ของ Database Subsystem ระบบฐานข้อมูลของระบบงานบุคลากร ประกอบด้วย Database จำนวน 2 ชุด (ชุดใช้งานจริง และชุดBackup) ซึ่งทั้ง 2 Database มีลักษณะและข้อมูลเหมือนกัน และฐานข้อมูลทั้ง 2 ชุดถูกเชื่อมโยงกันด้วย Connection เช่น Internet เป็นต้น

NODE :

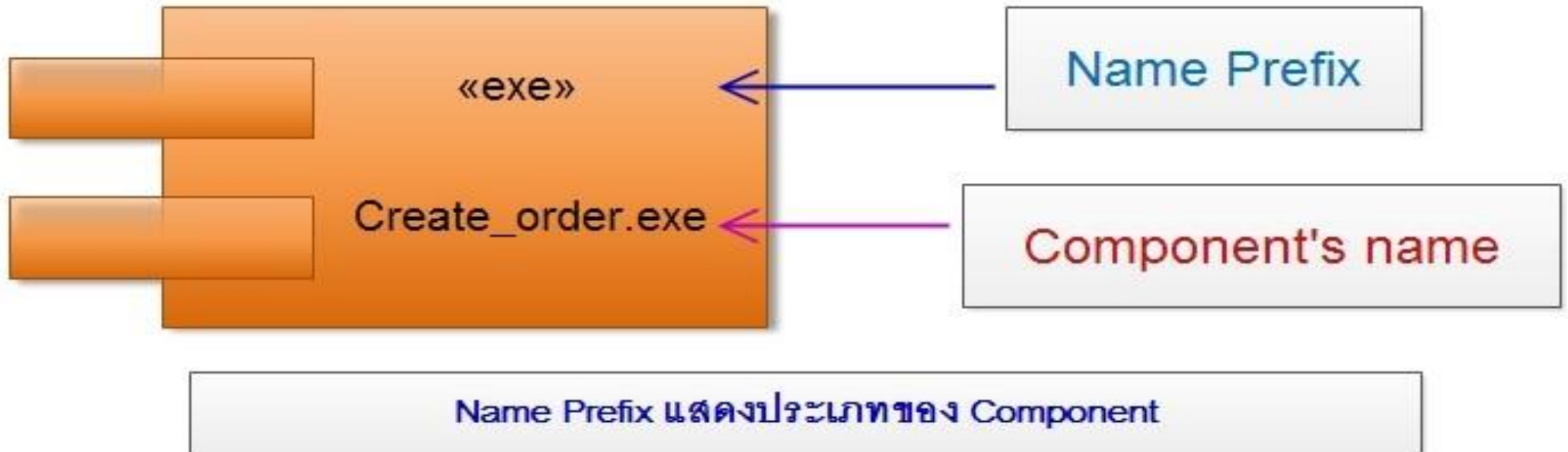
- ในขั้นตอน Design ใช้ Node เป็นตัวแทน H/W (Processor) สามารถประมวลผล หรือจัดเก็บข้อมูล
- แต่ละ node มี S/W อย่างต่ำ 1 ตัววางอยู่ (deploy) หรือ execute บนnode



Node เป็น Unique Identity และมี Path name แสดงประเภทของ HW ที่ Node วางอยู่ (Path name เขียนในรูปแบบของ Stereotype)

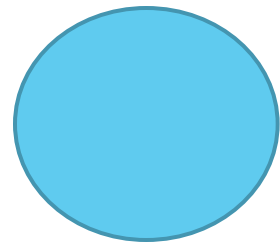
Component :

- เป็นสัญลักษณ์เป็นตัวแทนของ S/W โดย component ถูกวางบน node ใด node หนึ่งเสมอ
- มีหน้าที่ทำให้เกิดงานของ Interface ต่างๆ เรียกว่า **Interface Realization**
- Component มีหลายรูปแบบ เช่น exe, java, php, DLL



Interface :

-เป็น class ทำหน้าที่รวบรวมกลุ่มของ Operation/Method ที่ทำให้ทราบว่า component มี operation ใดบ้างที่ให้ component อื่นมาเรียกใช้



IBrowser



ชื่อ Interface

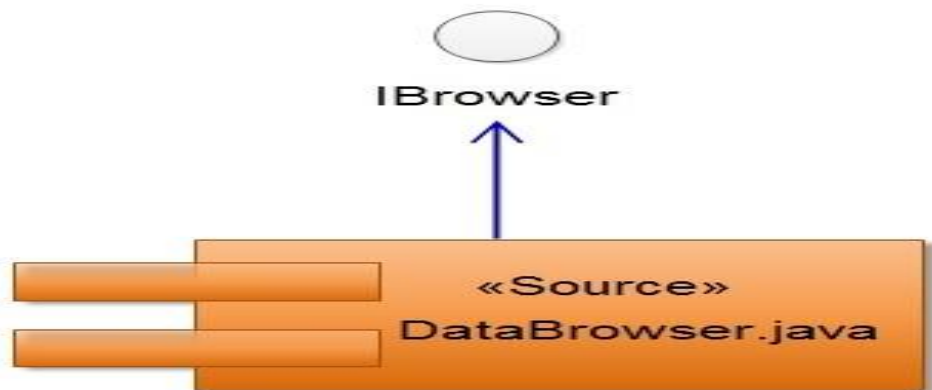
ความสัมพันธ์ระหว่าง Component กับ Interface มี 2 แบบ

1) Realization

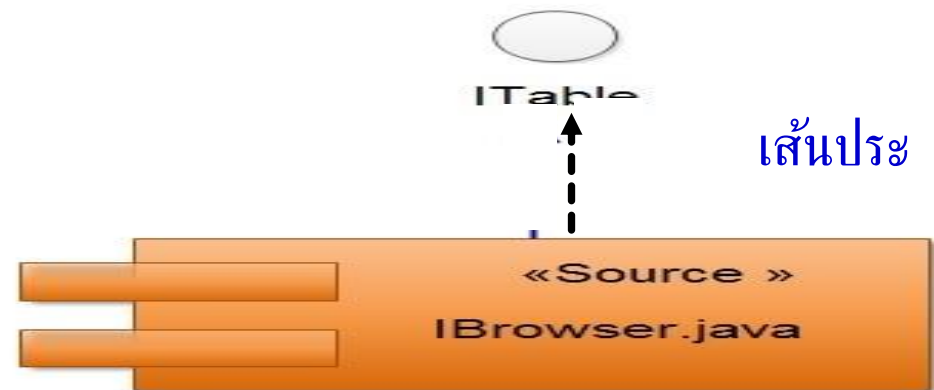
คือ กระบวนการที่ Component ทำหน้าที่ **Implement** รายละเอียดของ Method ต่างๆ ที่ประกาศไว้ใน Interface ตัวใดตัวหนึ่ง

-เรียก Interface ที่ถูก Realize ด้วย Component นี้ว่า **Export Interface**

-ใช้ ลูกศรเส้นทึบ จาก Component ไป Interface



สัญลักษณ์ความสัมพันธ์แบบ
Realization



สัญลักษณ์ความสัมพันธ์แบบ
Dependency

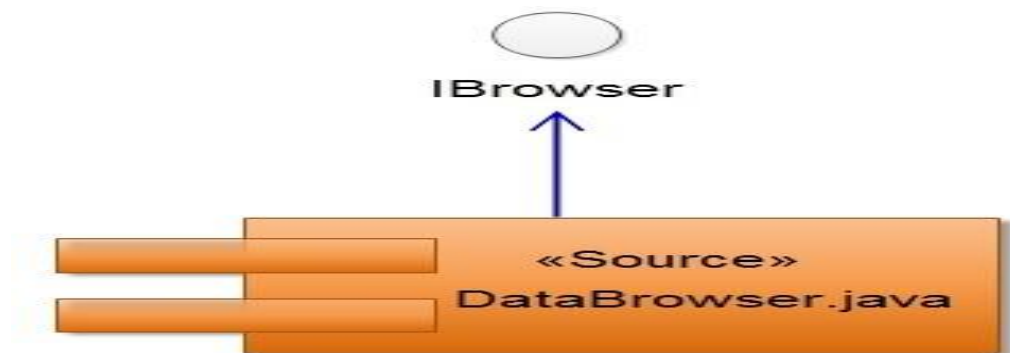
ความสัมพันธ์ระหว่าง Component กับ Interface มี 2 แบบ

2) Dependency

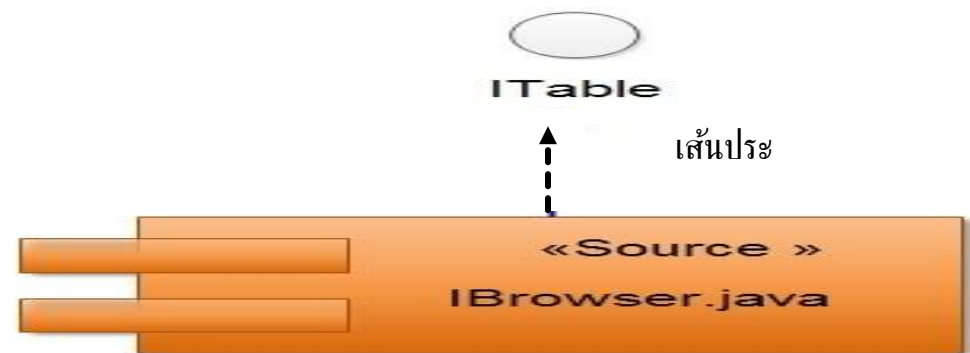
คือ กระบวนการที่ Component นำ Method ที่ประกาศไว้ใน Interface ตั้งแต่ 1 ตัวขึ้นไป เข้ามาใช้ประโยชน์ใน Component นั้น

-เรียก Interface ที่ถูกดำเนินการด้วยกระบวนการ Dependency นี้ว่า **Import Interface**

-ใช้ ลูกศรเส้นประ จาก Component ไป Interface



สัญลักษณ์ความสัมพันธ์แบบ
Realization



สัญลักษณ์ความสัมพันธ์แบบ
Dependency

Type of Component : (แบ่งตามลักษณะและประโยชน์ในการใช้งาน) มี 5 กลุ่ม คือ

1) Executable (EXE) Component : Component อยู่บน node ใดๆ ที่สามารถประมวลผล บน node นั้นได้

2) Library Component : Component อยู่บน node ใดๆ ไม่สามารถประมวลผลได้ แต่เป็นที่เก็บของ object ต่างๆ ที่สามารถถูกนำไปใช้งานโดย Executable components ได้

Type of Component : (แบ่งตามลักษณะและประโยชน์ในการใช้งาน)

มี 5 กลุ่ม คือ (ต่อ)

3) Table Component : Component เป็นที่เก็บของ Database Table มีโครงสร้างและแบบการปฏิบัติเพื่อจัดการกับข้อมูลที่ชัดเจนแน่นอน

4) File Component : Component เป็นตัวแทนของเอกสารเพื่อเก็บ Data และ Source Code

5) Document Component : Component เป็นตัวแทนของเอกสารอื่นๆ (Documents) ไม่ใช่เก็บ Data และ Source Program

การเขียนและออกแบบ Component Diagram

สัญลักษณ์ที่ใช้ในการเขียน Component Diagram และตัวอย่างการเขียน Component Diagram จะมีลักษณะคล้ายคลึงกับ Class Diagram และ Use Case Diagram โดยสัญลักษณ์ที่ใช้สำหรับ Diagram ประเภทนี้ แบ่งออกเป็น 2 ส่วน คือ สัญลักษณ์แทน Component และสัญลักษณ์แทน Link หรือ Relationship ระหว่าง Component

สัญลักษณ์ที่ใช้แทน Component ต่างๆ มีดังนี้

1. โปรแกรมที่สามารถนำไปประมวลผลได้ (Executable Program)
2. ไฟล์ หรือ Source Code
3. ฐานข้อมูล (Database)
4. ชุดข้อมูลในฐานข้อมูล (เช่น Table ใน Relational Database)

สัญลักษณ์ที่ใช้แทน Links มีดังนี้

1. สัญลักษณ์การเรียกใช้งานหรือการขึ้นต่อกัน (Calls, Uses หรือ Dependency)
2. สัญลักษณ์แสดงการเชื่อมโยงหรือเชื่อมต่อ (General Connection) เช่น การเชื่อมโยงด้วยระบบเครือข่าย หรือ Internet
3. สัญลักษณ์แสดงการเป็นส่วนประกอบ (Aggregation)



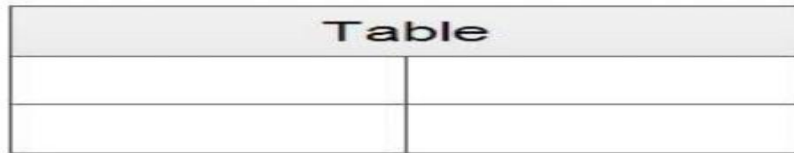
Executable Program



Page of File



Database



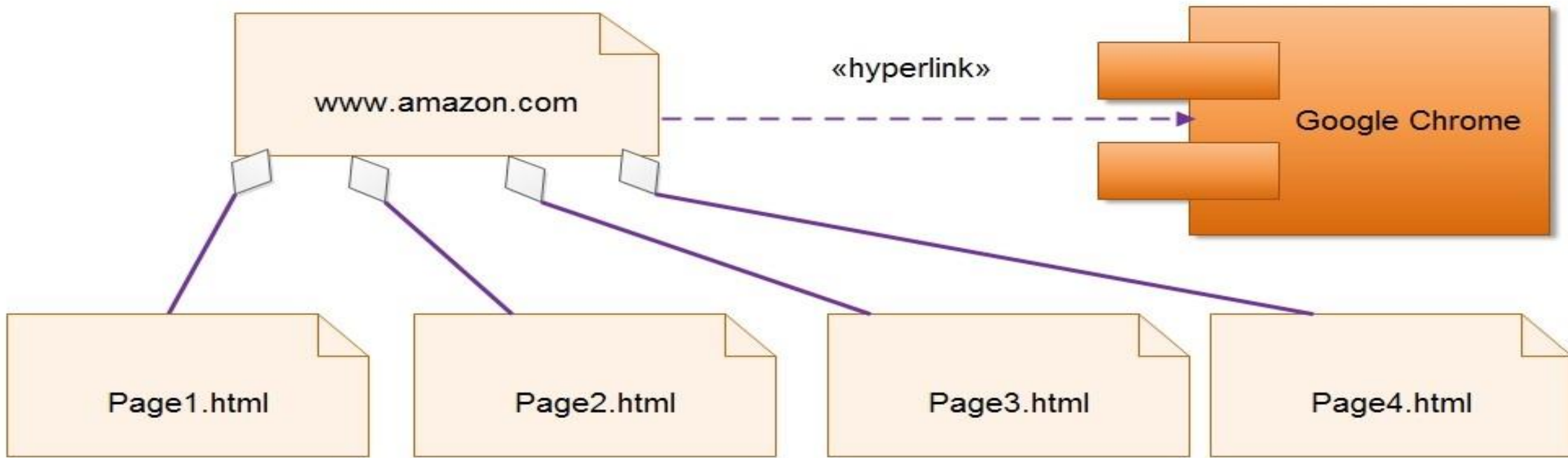
Dependency, Calls or Uses



General Connection

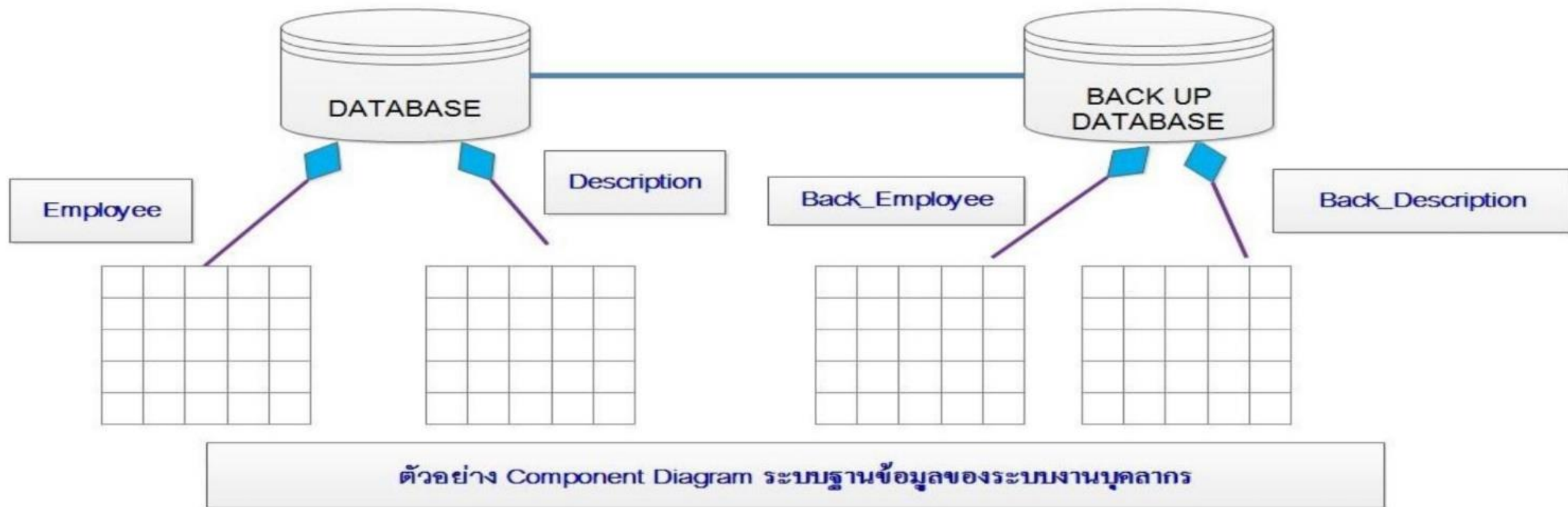


Composition or Aggregation



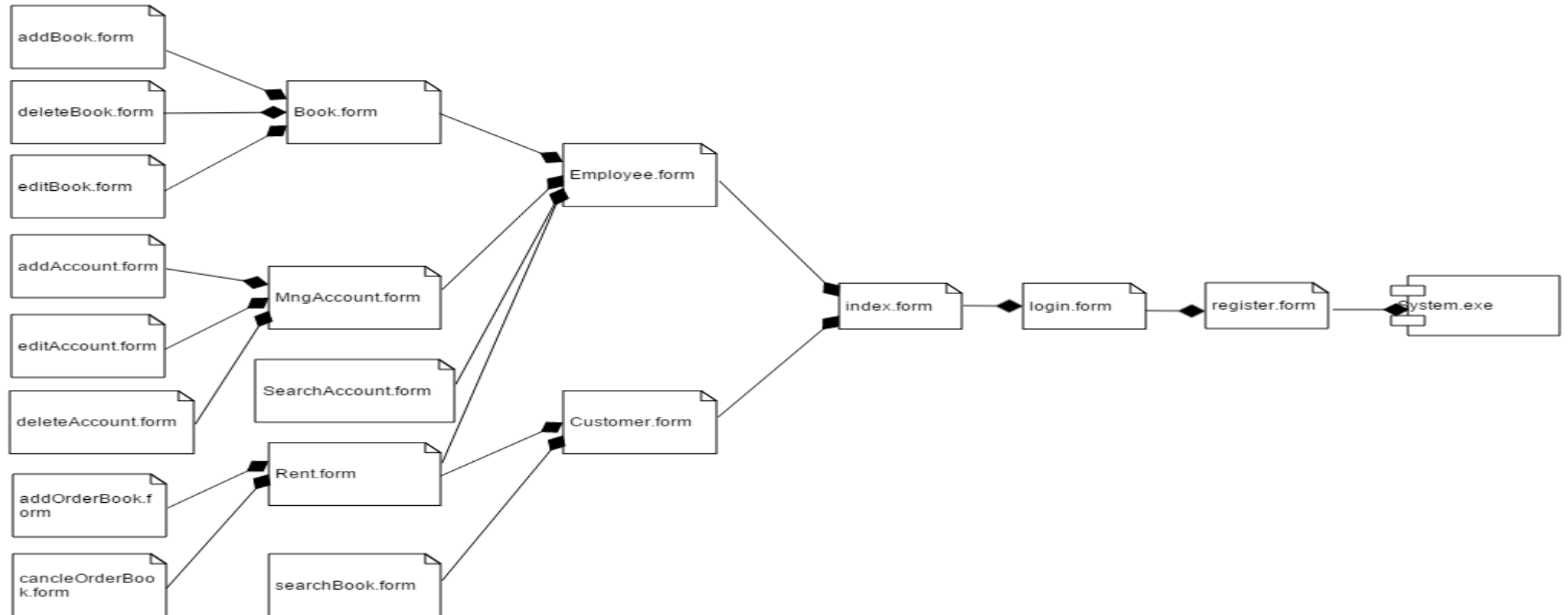
ตัวอย่าง Component Diagram ระบบเรียกดูข้อมูลจาก Web Site (Web Browsing)

Ex. 1 : Component Diagram ของ Web Browsing System ดังรูป ซึ่งระบบประกอบด้วย Web Site ชื่อ `www.amazon.com` ซึ่งประกอบด้วย 4 web page ทั้งหมดจัดเป็น File ซึ่งจะต้องอ่านด้วย (หรือ ทั้งหมดขึ้นอยู่กับ หรือถูกเรียก โดย) Google Chrome ซึ่งเป็น Executable Program

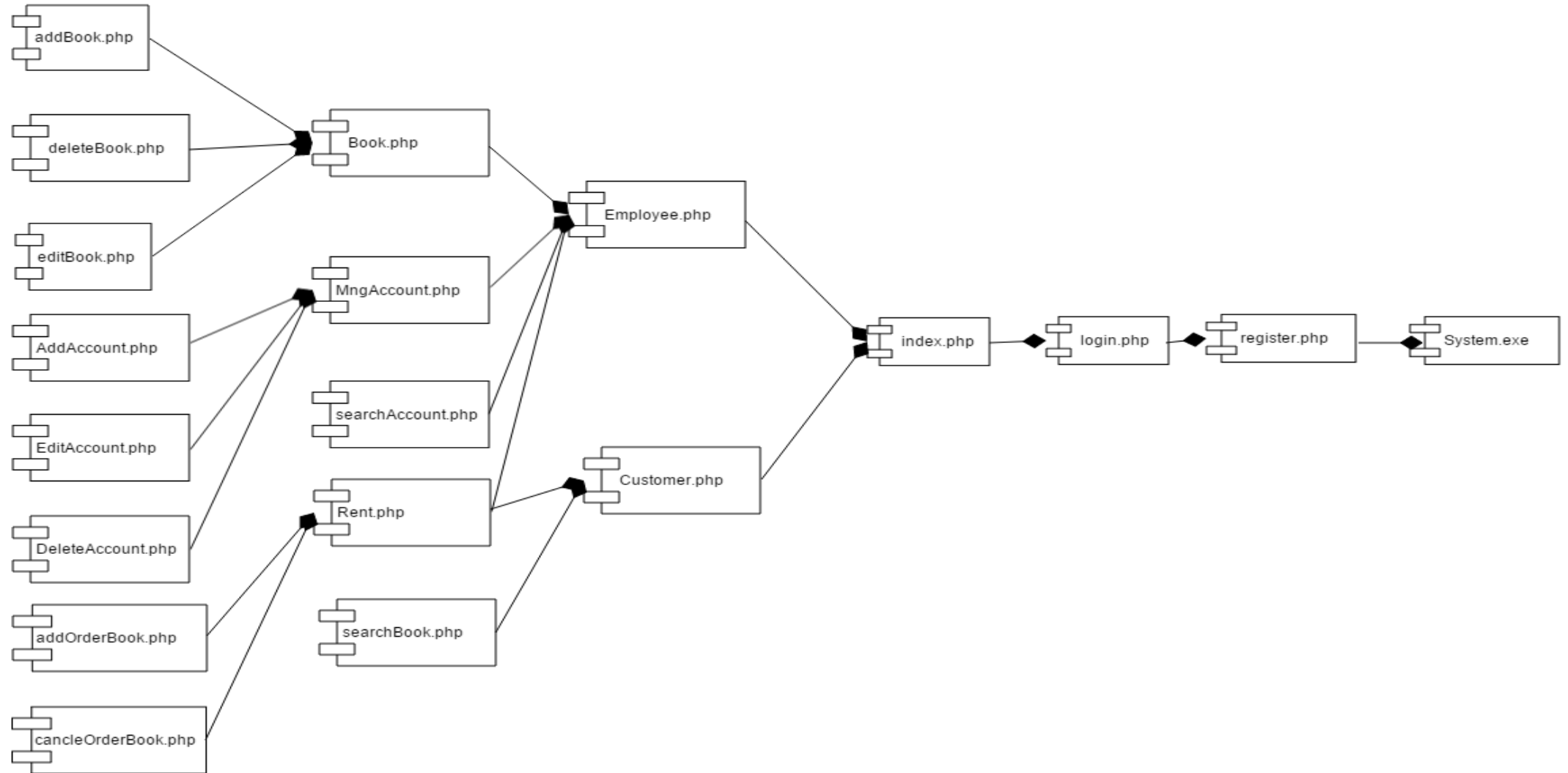


Ex. 2 : เป็น Component Diagram ของ Database Subsystem ระบบฐานข้อมูลของระบบงานบุคลากร ประกอบด้วย Database จำนวน 2 ชุด (ชุดใช้งานจริง และชุดBackup) ซึ่งทั้ง 2 Database มีลักษณะและข้อมูลเหมือนกัน และฐานข้อมูลทั้ง 2 ชุดถูกเชื่อมโยงกันด้วย Connection เช่น Internet เป็นต้น

ตัวอย่าง Presentation Logic Subsystem



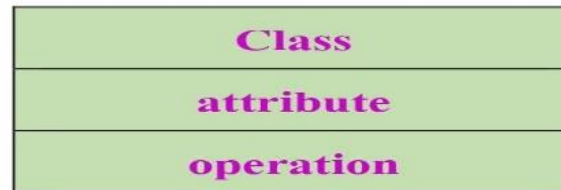
ตัวอย่าง Working Logic Subsystem



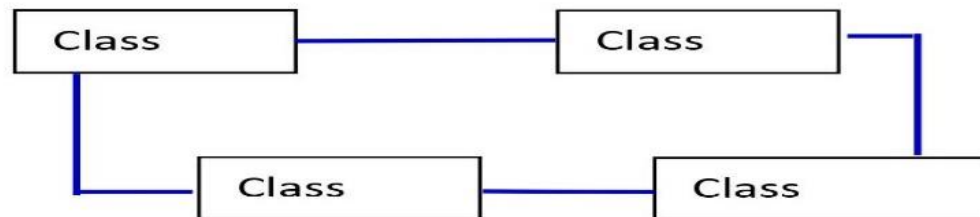
System Architecture

การออกแบบโครงสร้างภายในระบบมี 3 ระดับ คือ

1. ระดับรายละเอียดการทำงาน มีขอบเขตการทำงานภายใน Object โดยระบุรายละเอียดภายใน Object นั้นๆ ได้แก่ Algorithm ,ข้อมูล (เช่น order-id, order-date,...), Function (โครงสร้าง, parameter ที่ต้องการ ,) ภายใน Object



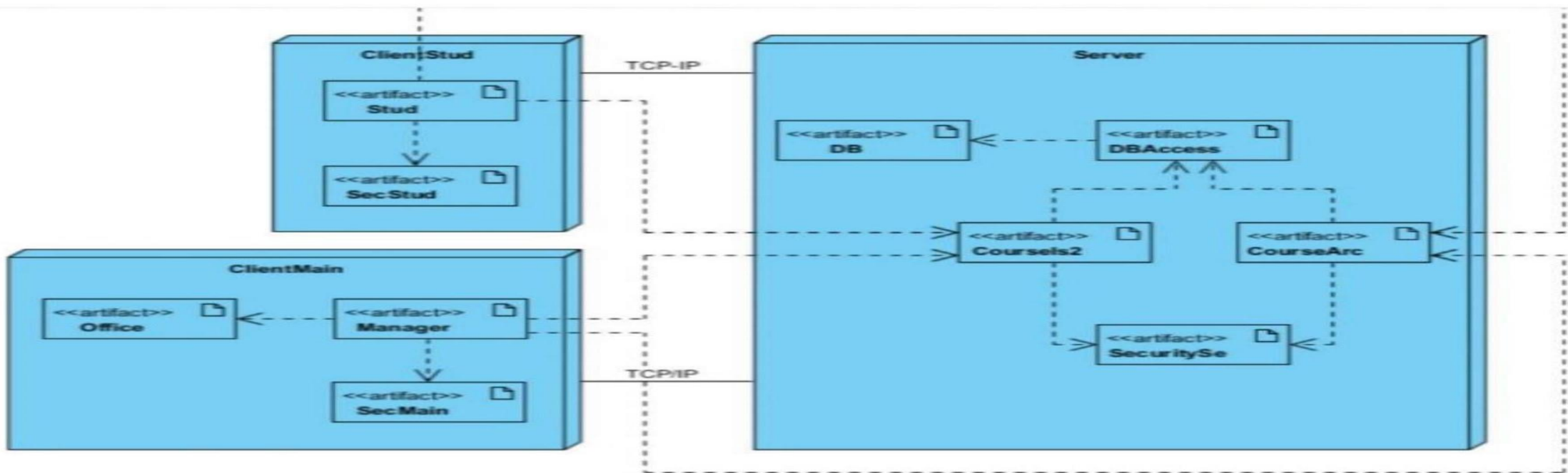
2. ระดับโครงสร้างการทำงาน เกี่ยวข้องกับโครงสร้างการทำงานระหว่าง Class กับ Object ที่มีอยู่ในระบบ เพื่อให้ตรงตามความรับผิดชอบของระบบ



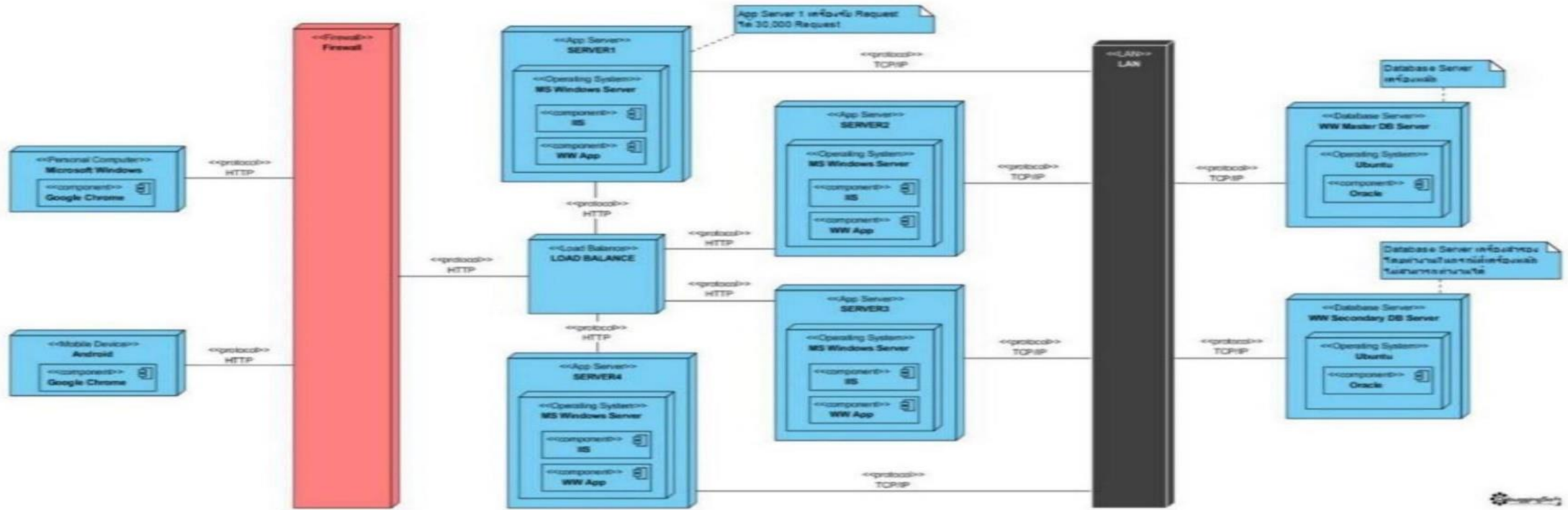
3. ระดับสถาปัตยกรรม ใช้ **Deployment Diagram** แสดงสถาปัตยกรรม มีขอบเขตการทำงานในระดับของ System และระดับ Processor โดยระบุรายละเอียดได้แก่ จำนวนและชนิดของ Processor, กลุ่มของ Object ที่ทำงานบน Processor, สื่อและProtocol ที่ใช้สื่อสารระหว่าง Processor, Model ในการทำงานแบบพร้อมๆกัน เป็นต้น

Deployment diagram

Deployment diagram เป็น Structured Diagram ที่แสดงสถาปัตยกรรมในลักษณะ physical architecture แบบ run-time (เวลาที่ใช้ในการทำงาน) ของโปรเซสเซอร์ แสดงโครงสร้างการใช้ทรัพยากร Hardware และ Software ซึ่งรวมไปถึงการแสดงความสัมพันธ์ ของ อุปกรณ์ในระบบรวมถึง middleware ที่ทำหน้าที่เชื่อมต่อกับคอมพิวเตอร์อื่น ๆ ซึ่งเป็นส่วนหนึ่ง ของการ วิเคราะห์ และออกแบบ ระบบคอมพิวเตอร์ เชิงวัตถุ



ตัวอย่าง Deployment Diagram



ตัวอย่าง Deployment Diagram ภาพจาก <https://naiwaen.debuggingsoft.com/2018/05/uml-deployment-diagram-with-load-balance-firewall-high-availability>

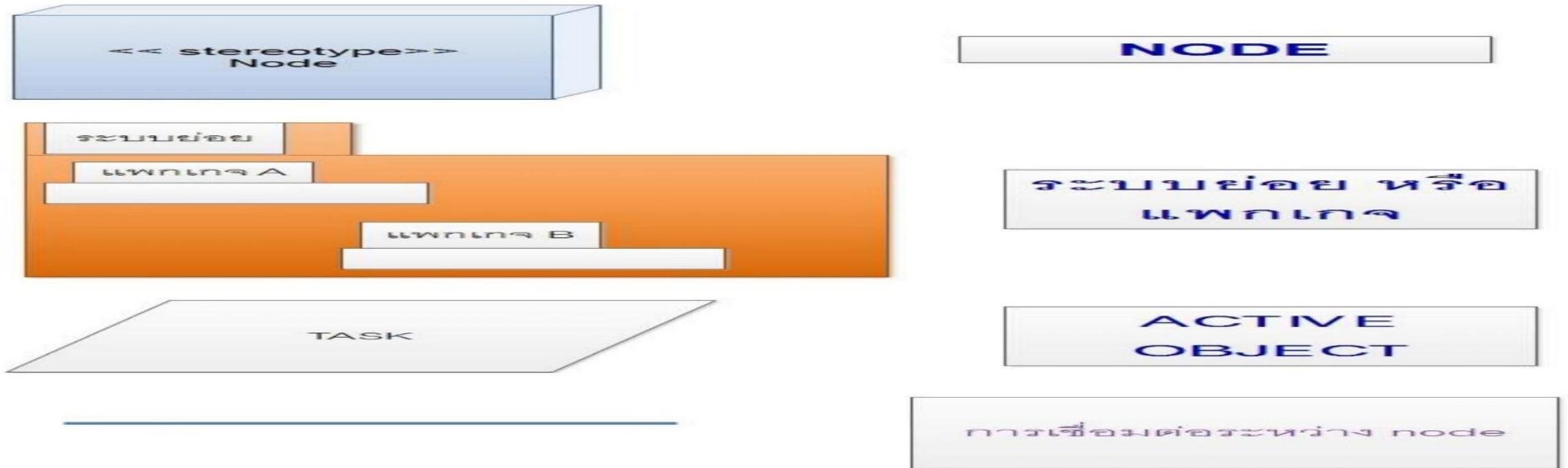
- ถือว่าเป็นการกำหนดรายละเอียดขั้นตอนสุดท้าย ของระบบ ที่เรียกว่า System Topology ซึ่งประกอบด้วยรายละเอียดของ Hardware และ Software ที่ Run อยู่ในหน่วยนั้น ๆ
- deployment Diagram แสดงเป็นภาพของ Node ที่เชื่อมต่อกัน มีการติดต่อกันระหว่างกัน บาง Node อาจมี component instances ที่แสดงว่า Component runs อยู่บน node นั้น ๆ และบาง Component อาจประกอบด้วย Object ได้ แสดงว่า Object นั้นเป็นส่วนหนึ่งของ Component

วัตถุประสงค์ของการใช้ Deployment diagrams

- 1) แสดงรายละเอียดที่เกี่ยวกับการติดตั้งระบบในรูปของ H/W และ S/W ที่เกี่ยวข้องกัน
- 2) แสดงรายละเอียดของ dependencies ที่ระบบขึ้นอยู่กับระบบเดิมที่มีอยู่ หรือวางแผนสำหรับ production environment.
- 3) แสดงให้เห็นถึง configuration หลักของโปรแกรมประยุกต์ทางด้านธุรกิจ.
- 4) ออกแบบ Configuration ของ Hardware และ Software ของ embedded system (เป็นระบบที่เน้นการใช้ Software เพื่อทำงานอย่างใดอย่างหนึ่งโดยเฉพาะ และไม่เน้นการสื่อสารระหว่างหน่วยประมวลผลอื่นๆ เช่น ใช้ Program เพื่อควบคุมการทำงานของเครื่องกล)
- 5) แสดงให้เห็นถึงโครงสร้างพื้นฐานของ hardware/network ขององค์กร
- 6) ใช้เป็นข้อมูลในการพิจารณา Performance, Maintainability, Scalability, Portability

คุณสมบัติของ Deployment

- deployment ไดอะแกรมอาจใช้ในการแสดงให้เห็นว่า Componentใดที่ Run อยู่กับ nodes ใด และแสดงการเคลื่อนย้ายของ Component จาก node หนึ่งไปยัง node อื่น ๆ หรือ Objectจาก Component หนึ่ง ไปยัง Component อื่น ๆ อาจนำเสนอได้โดยใช้สัญลักษณ์ «becomes» stereotype ร่วมกับความสัมพันธ์แบบ dependency



Communication Path ใช้แสดง การเชื่อมต่อระบบ ระหว่าง อุปกรณ์ โดยระบุ รูปแบบการเชื่อมต่อ เช่น << Lan >> << TPC/Protocal >>





- Nodes หนึ่งอาจเชื่อมต่อได้กับ nodes อื่น ๆ ความเกี่ยวข้องดังกล่าวจะแสดงให้เห็นถึงเส้นทางในการติดต่อกันระหว่าง node ความเกี่ยวข้องกันอาจนำเสนอผ่าน stereotype เพื่อช่วยระบุเส้นทางในการติดต่อระหว่างกัน
- การสร้าง deployment model จะเหมาะสำหรับ applications ที่มีการนำไปใช้ในหลาย ๆ machine รูปแบบของ deployment model ที่เหมาะสมที่จะทำงานบน thin-client network computer ที่มีการเข้าถึง database server โดยตรง ส่วนแบบอื่นจะได้แก่ระบบ customer service system ที่ถูกนำไปใช้ในสถาปัตยกรรมแบบ a distributed object
- นอกจากนั้น deployment models ยังเหมาะสมสำหรับการออกแบบ embedded systems ที่ต้องการแสดงให้เห็นถึงวิธีการที่ H/W และ Software Component ทำงานร่วมกัน

ข้อกำหนด

การทำ System Architecture Design หรือการออกแบบระบบ Hardware ด้วย Deployment Diagram นั้น ต้องมีความสัมพันธ์กับ Application Architecture Design ซึ่งออกแบบด้วย Component Diagram คือ แต่ละ Software Component ต้องอยู่ภายใน Hardware Module ใด Module หนึ่งเสมอจะต้องไม่มี Software Component ใดขาดหายไป

ดังนั้นเมื่อสร้าง Deployment Diagram สิ่งที่ต้องทำไปพร้อมๆ กันก็คือ พิจารณาว่าจะนำ Software Component ไปไว้ใน Hardware Module ใด และต้องพิจารณาด้วยว่า เทคโนโลยีหรือรูปแบบของ Hardware หรือระบบ Network ที่จะใช้นั้น เหมาะสมกับระบบงานที่เราจะสร้างหรือไม่

องค์ประกอบหลัก ในการออกแบบระบบ ด้วย Deployment Diagram

Hardware

1. Server Computer
2. Client Computer
3. Connecting Network

Soft ware

1. Data Stored แหล่งเก็บข้อมูล
2. Data Access Logical
3. Application Logical

สัญลักษณ์ในการออกแบบ Deployment Diagram

1) NODE



Node

เป็นสัญลักษณ์ ที่ใช้แสดงแทน Hardware ในแต่ละ อุปกรณ์

Application Name

เป็นสัญลักษณ์ ที่ใช้แสดงส่วนของ Software หรือ Database ที่ใช้ภายใน Node

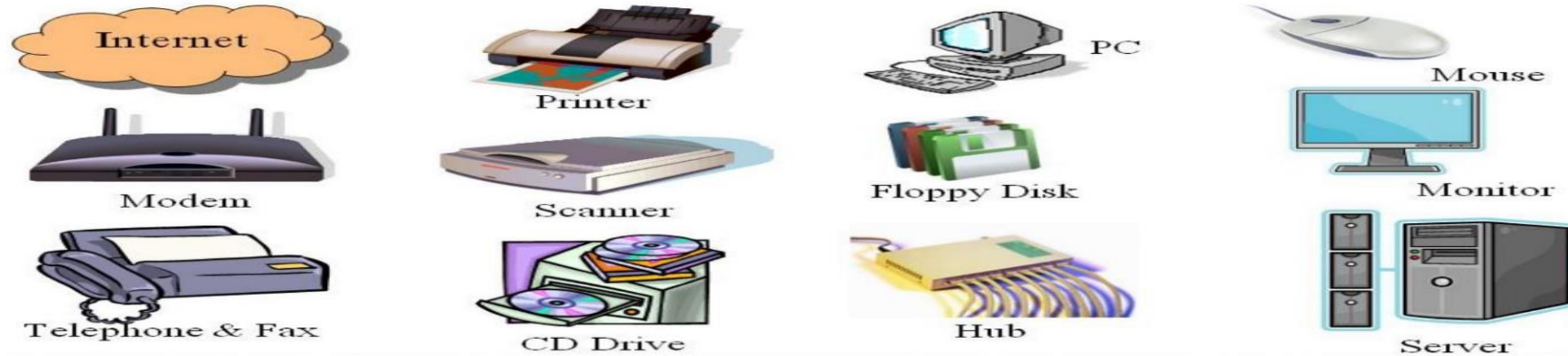
Application in Node

ตัวอย่าง แสดง Hardware: Server มี DBMS เช่น Oracle อยู่ภายใน Node

2 Communication Path ใช้แสดง การเชื่อมต่อระบบ ระหว่าง อุปกรณ์ โดยระบุ รูปแบบการเชื่อมต่อ เช่น << Lan >> << TPC/Protocol >>

<< Communication Path >>

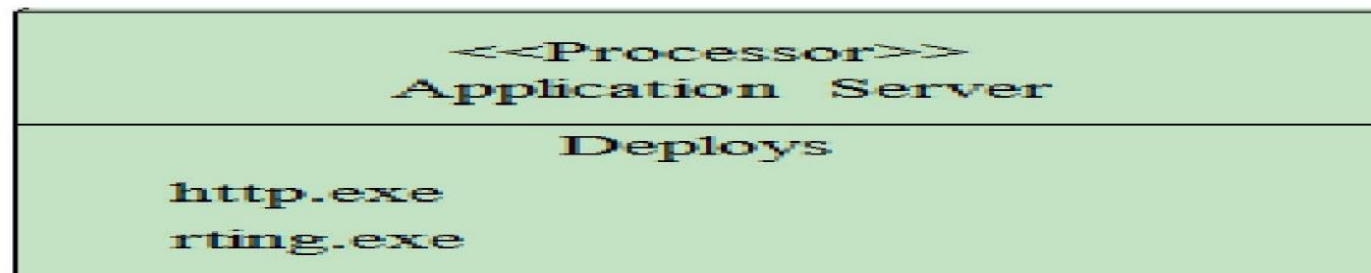
3. สัญลักษณ์อื่นๆ เพิ่มเติม ดังรูป



หลักการในการเขียน Hardware Module มี 2 ส่วนคือ

1. ส่วนบนให้ระบุประเภทของ Hardware นั้นโดยระบุในเครื่องหมาย <<..... >> และเขียนชื่อของ Hardware นั้นไว้
2. ส่วนล่าง (จะมีหรือไม่ก็ได้) ให้ระบุว่า Hardware นั้นมี Software Component ใดอยู่บ้าง ซึ่ง Software Component ที่จะระบุใน Hardware Module ได้นั้น ได้จากขั้นตอนการทำ Application Architecture Design

ตัวอย่างจากรูป จะพบว่า Hardware Module นี้ มีประเภทเป็น Processor ซึ่งมีชื่อ Application Server โดย Software Component ที่อยู่ภายในประกอบไปด้วย http.exe และ rting.exe ซึ่งทั้งสองถือเป็น Executable Program

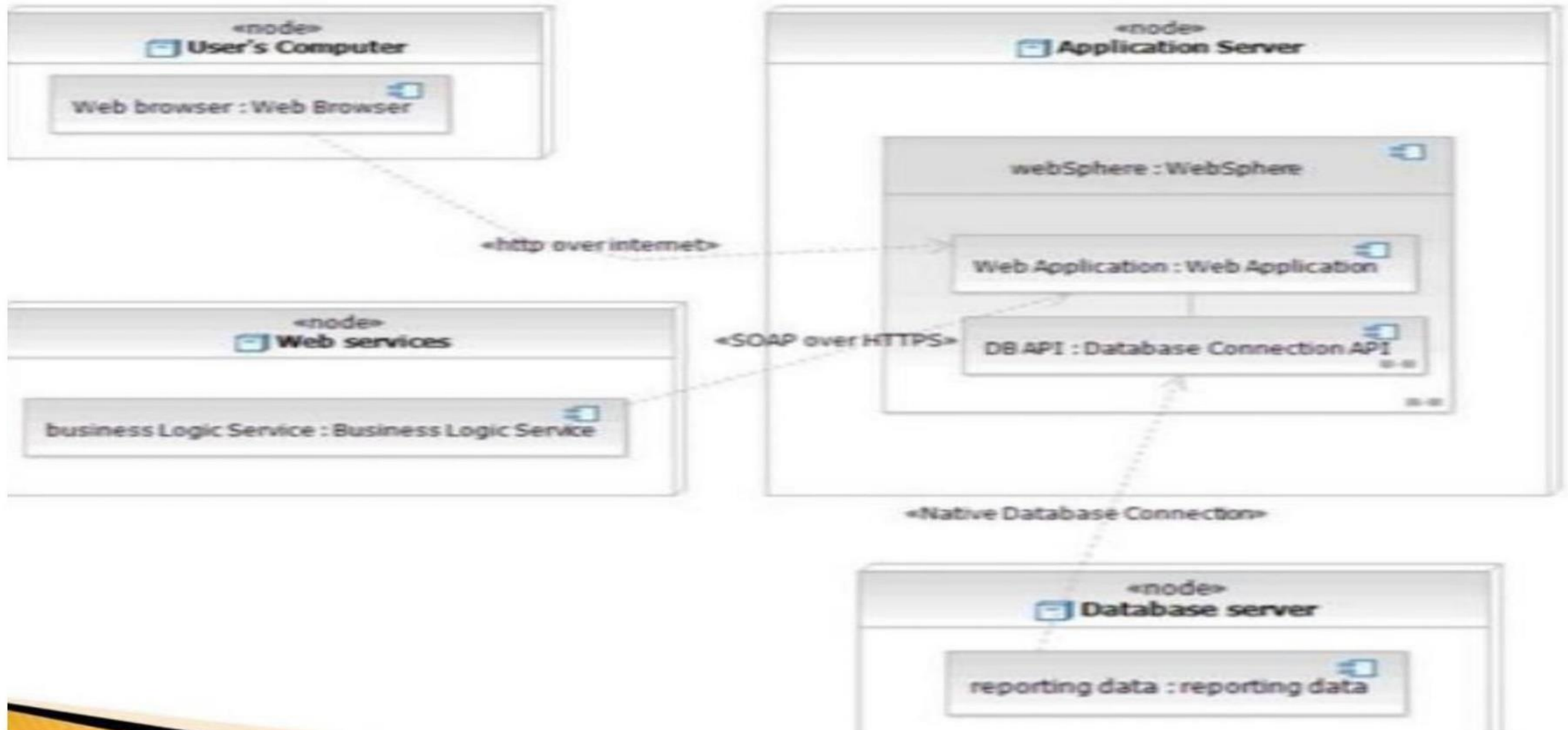


การออกแบบ Deployment Diagram มี 4 รูปแบบ

1) Embedded system (เป็นระบบที่เน้นการใช้ Software เพื่อทำงานอย่างใดอย่างหนึ่งโดยเฉพาะ และไม่เน้นการสื่อสารระหว่างหน่วยประมวลผลอื่นๆ เช่น ใช้ Program เพื่อควบคุมการทำงานของเครื่องกล)

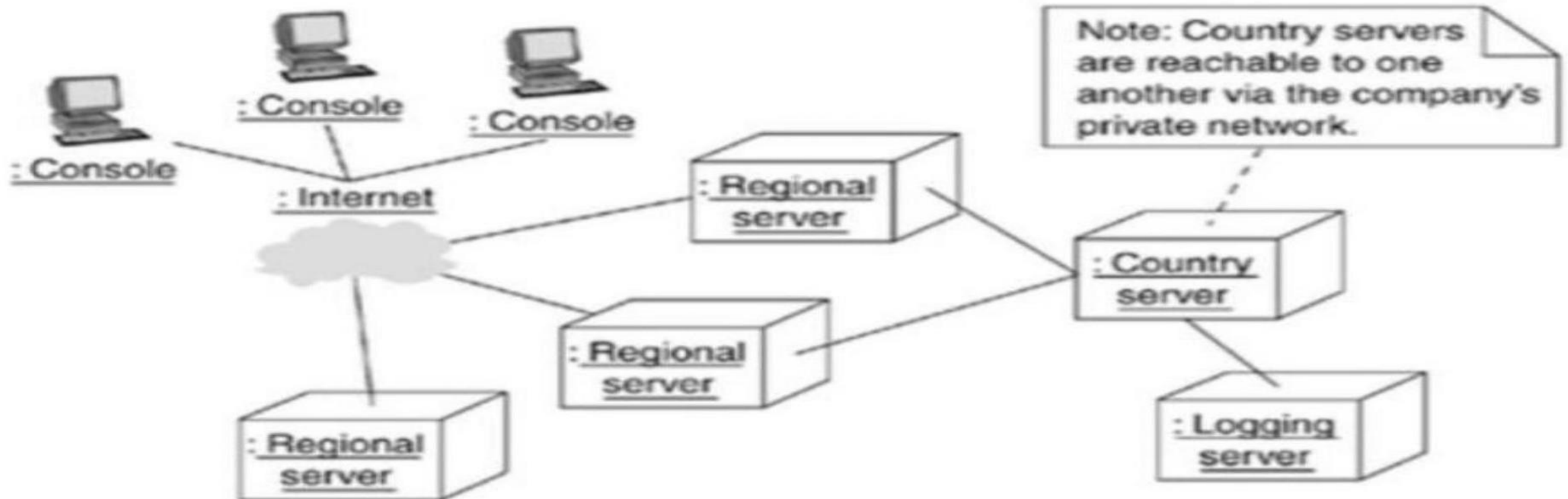
2) Stand alone System (ไม่นิยม) เป็นระบบที่มี Processor ประมวลผลเพียง 1 ระบบต่อผู้ใช้งานแต่ละคน โดย I/O device เพียงแต่ รับ-ส่ง คำสั่งจาก Processor เท่านั้น จึงเรียก I/O device นี้ว่า Dump Terminal

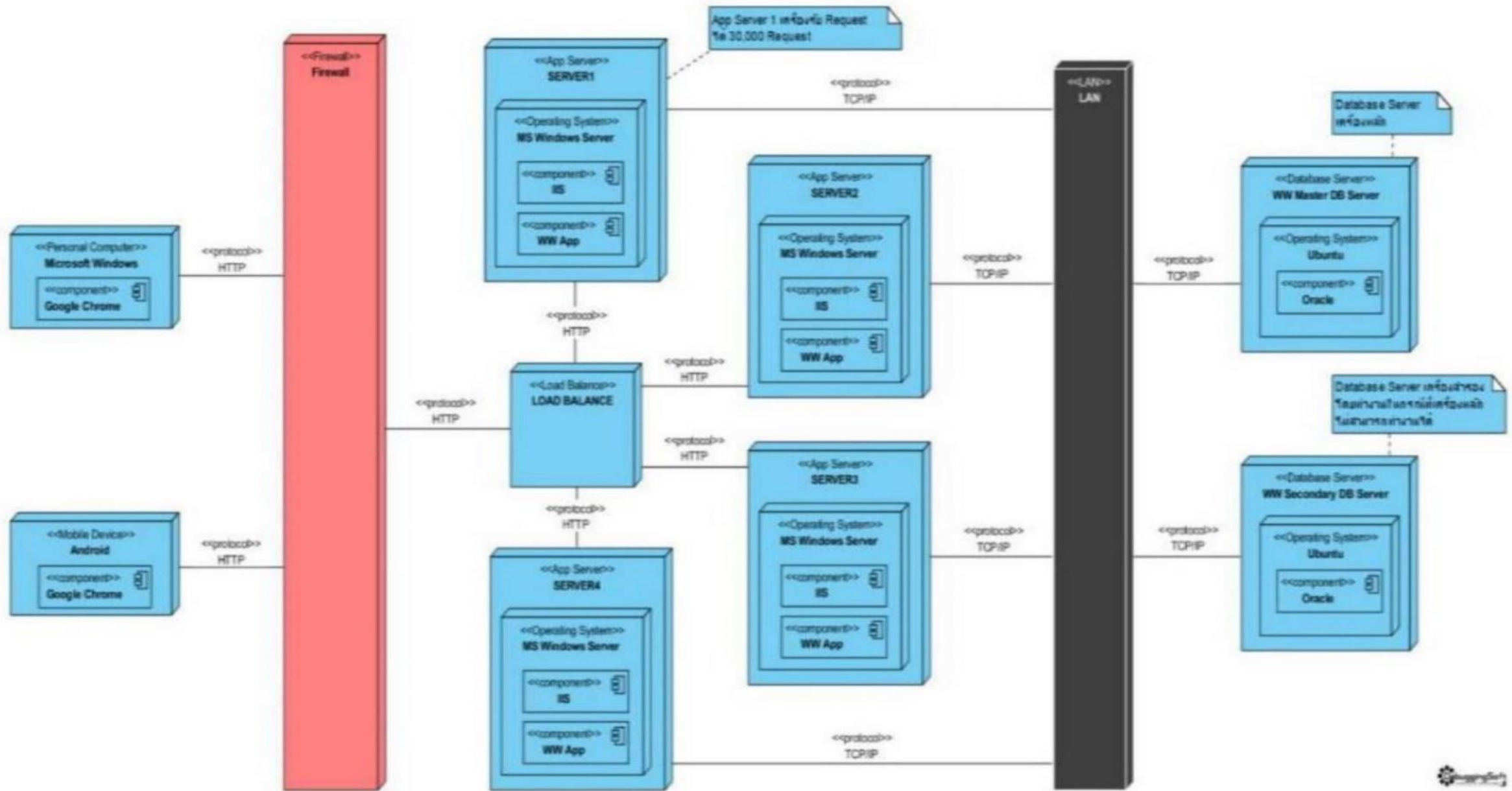
3) Client/Server System



Client Server ภาพจาก <http://www.itsci.mju.ac.th/sayan>

4) Distributed System มี Server และ Client แต่มี User ทำงานได้พร้อมกันหลายคน โดยมากมีหลาย Processor เพื่อรองรับงานงานด้านอื่นๆได้





ตัวอย่าง Deployment Diagram ภาพจาก <https://naiwaen.debuggingsoft.com/2018/05/uml-deployment-diagram->

Deployment Environment

- ▶ **Deployment environment เพื่อรองรับ Application S/W**
- ▶ **Deployment environment ได้แก่**
 - ▶ **Hardware**
 - ▶ **System software**
 - ▶ **การสร้าง Network**

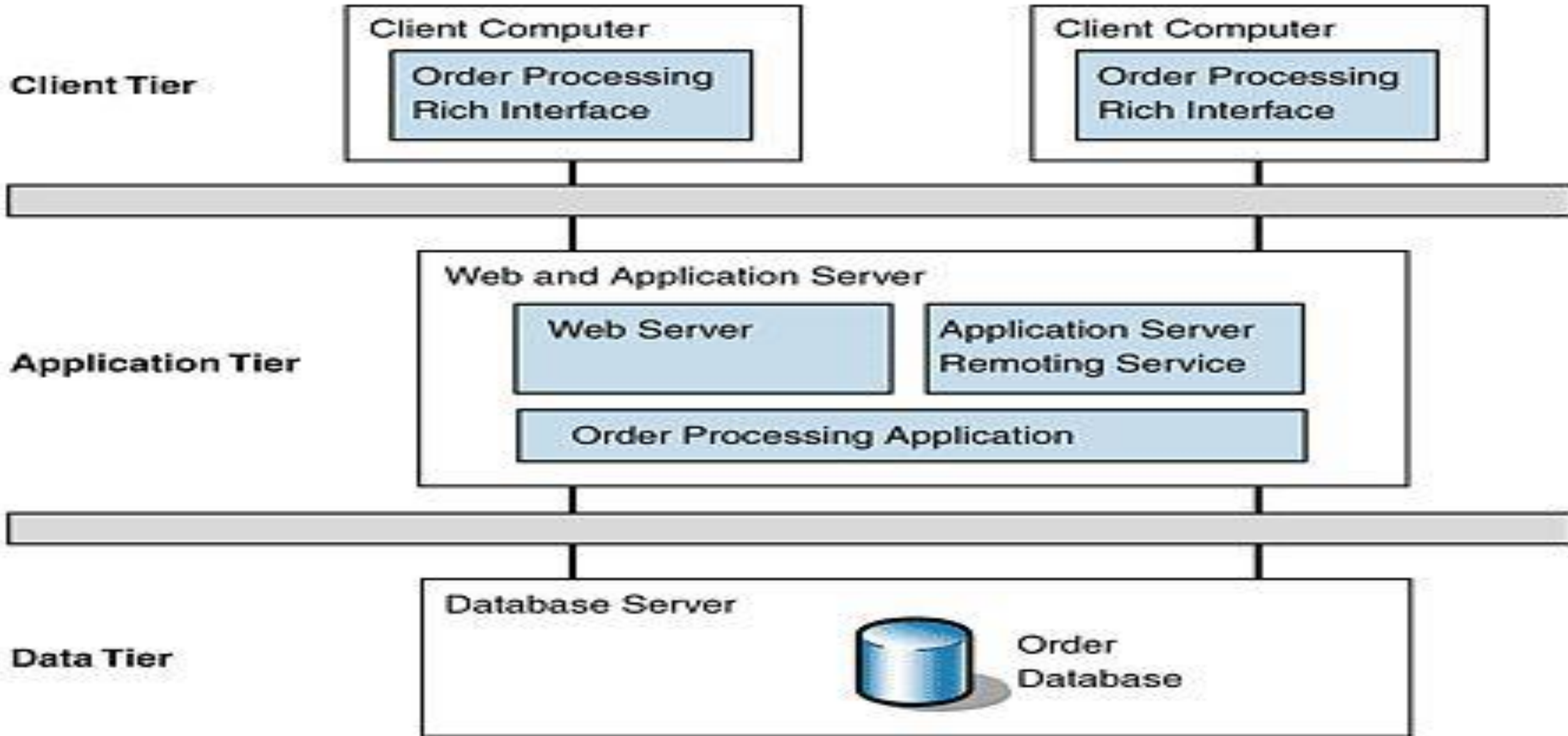
Application Architecture มี 4 รูปแบบที่นิยม ได้แก่

- **Client/server architecture**
- **Three-layer client/server architecture**
- **Web services architecture**
- **Internet and Web-based application architecture**

Client/Server Architecture

- ▶ **Client/server divides programs into two types**
- ▶ **Server – manages information system resources or provides well-defined services for client**
- ▶ **Client – communicates with server to request resources or services**
- ▶ **Advantage – deployment flexibility**
 - ▶ **Location, scalability, maintainability**
- ▶ **Disadvantage – complexity**
 - ▶ **Performance, security, and reliability**

Example of Three-layer client/server architecture



Internet and Web-Based Application Architecture

- ▶ **Web is complex example of client/server architecture**
- ▶ **Can use Web protocols and browsers as application interfaces**
- ▶ **Benefits**
 - ▶ **Accessibility**
 - ▶ **Low-cost communication**
 - ▶ **Widely implemented standards**

WEB SERVICES, SERVICE ARCHITECTURES

- ▶ Web Services คือกลุ่มของส่วนประกอบ Software หลายชนิดที่ร่วมมือกัน เพื่อแลกเปลี่ยนข้อมูลกับ Software อื่นๆ โดยใช้ ภาษาและมาตรฐานสากล สำหรับการสื่อสารบน Web ระหว่างระบบคอมพิวเตอร์ที่ใช้ Operating System หรือ Programming Language ต่างกันได้ โดยอาศัย Web-Based Application Program เชื่อมโยงระบบคอมพิวเตอร์เข้าด้วยกัน ทำให้เราสามารถเรียกใช้ Component ใด ๆ ก็ได้ ใน ระบบ หรือ Platform ใด ๆ ก็ได้ บน Protocol HTTP ซึ่งเป็น Protocol สำหรับ World Wide Web หรือ อินเทอร์เน็ต อันเป็นช่องทางที่ได้รับการยอมรับทั่วโลกในการติดต่อสื่อสารกัน ระหว่าง Application กับ Application ในปัจจุบัน

WEB SERVICES, SERVICE ARCHITECTURES (ต่อ)

▶ ประโยชน์ของ Web Services

- ▶ 1. Web Services ช่วยให้การเข้าถึงข้อมูลสารสนเทศจากแอปพลิเคชันที่ต่างกันเป็นไปโดยง่าย โดยแอปพลิเคชันนั้นๆ สามารถเขียนด้วย Java และรันอยู่บน Sun Solaris Application Server หรืออาจจะเขียนด้วย C++ และรันอยู่บน Windows NT ซึ่งมาตรฐานของ Web Service ทำให้อินเทอร์เฟซของแอปพลิเคชันเหล่านี้ ถูกอธิบายโดย WSDL และทำให้อยู่ในมาตรฐานของ UDDI หลังจากนั้น จึงสามารถติดต่อสื่อสารถึงกันโดย XML ผ่าน SOAP อินเทอร์เน็ต
- ▶ 2. Web Services สามารถถูกเรียกใช้ภายในองค์กรเองหรือจากภายนอกองค์กร โดยผ่าน Fire Wall ดังนั้นจึงมีองค์กรใหญ่ๆ มากมาย กำลังพัฒนาระบบที่มีอยู่ของตน ให้เข้ากับ Web Services ซึ่งนับเป็นการลงทุนที่คุ้มค่า เนื่องจาก Web Services สามารถเพิ่มศักยภาพในการทำงานขององค์กร อีกทั้งลดค่าใช้จ่ายในการจัดการทรัพยากรขององค์กรได้อีกทางหนึ่ง

WEB SERVICES, SERVICE ARCHITECTURES (ต่อ)

ประโยชน์ของ Web Services (ต่อ)

3.Web Services สามารถใช้ร่วมกับ Web Application โดยส่งผ่านข้อมูลทางอินเทอร์เน็ตได้อีกด้วยซึ่งนับเป็นวิธีที่มีประสิทธิภาพในการติดต่อสื่อสารกับลูกค้าหรือหุ้นส่วน ถึงแม้จะต้องคำนึงถึงระบบรักษาความปลอดภัย และการจัดการรายการของข้อมูลอยู่ก็ตาม แต่ Web Services ได้ใช้มาตรฐานทั่วไปของ internet เรื่องดังกล่าวจึงนับเป็นเรื่องธรรมดาของการสื่อสารผ่านระบบอิเล็กทรอนิกส์

WEB SERVICES, SERVICE ARCHITECTURES (ต่อ)

Web Services ประกอบไปด้วย มาตรฐานหลัก 4 อย่าง ดังนี้

1. XML (Extensible Markup Language) เป็นภาษามาตรฐานที่ทุกระบบสนับสนุน ทำให้ข้อมูลที่มีโครงสร้างของภาษา XML จะถูกนำไปประมวลผลต่ออย่างอัตโนมัติได้อย่างง่ายดาย ภาษา XML จึงถูกนำมาใช้เป็นภาษามาตรฐานในการแลกเปลี่ยนข้อมูลของ Web Services

`<PASSENGERUNIT = "PASS"> 8 </PASSENGER>` หมายถึง Passenger 8 คน

WEB SERVICES, SERVICE ARCHITECTURES (ต่อ)

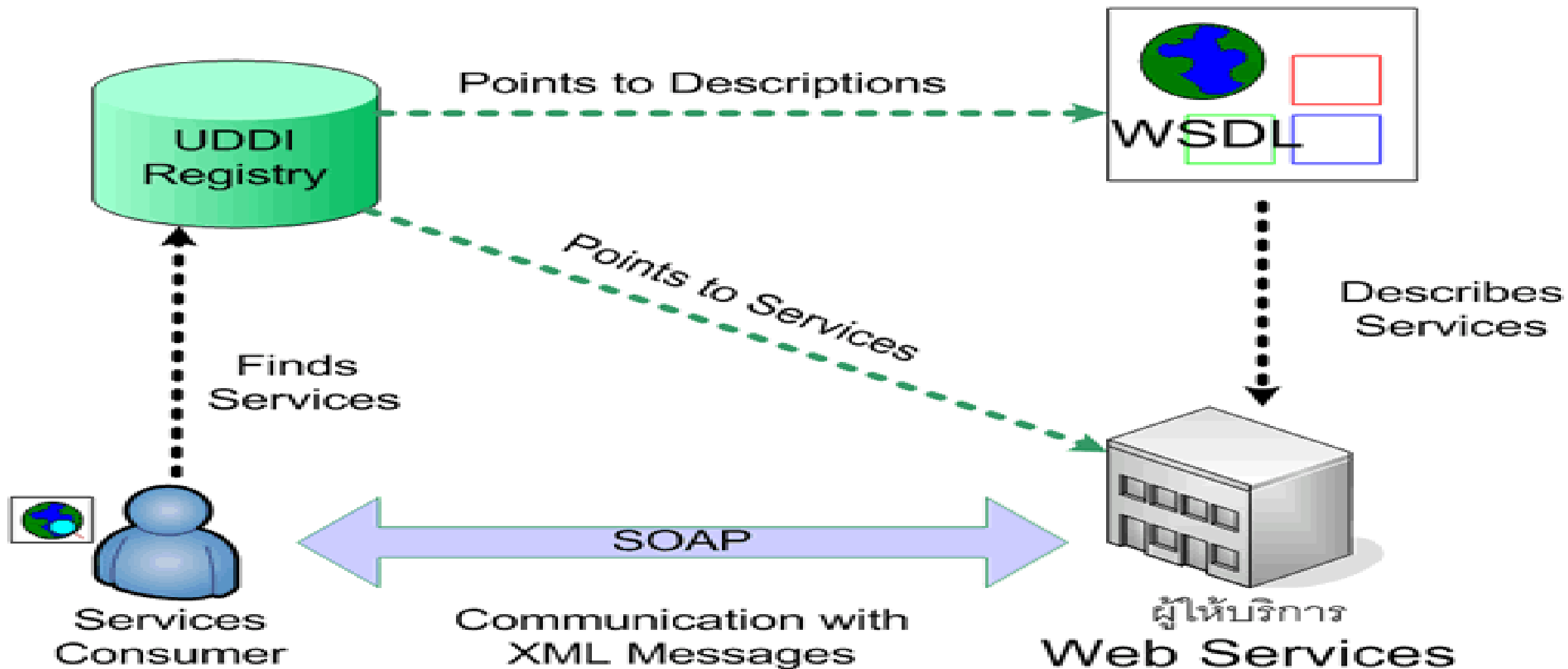
Web Services ประกอบไปด้วย มาตรฐานหลัก 4 อย่าง (ต่อ)

▶ 2. SOAP (Simple Object Access Protocol) เป็นมาตรฐานของเทคโนโลยี Distributed Objects โดยทำหน้าที่ส่งข้อมูลผ่านอินเทอร์เน็ต ในรูปแบบของ XML ทำให้เรียกใช้งานโปรแกรมข้ามระบบผ่านทางอินเทอร์เน็ตได้

▶ 3. WSDL (Web Services Description Language) เป็นภาษามาตรฐานที่ใช้สำหรับอธิบายการใช้งานโปรแกรมที่เปิดให้บริการ ซึ่งเขียนขึ้นตามแบบมาตรฐาน XML ดังนั้น WSDL จึงเป็นเสมือนคู่มือให้กับระบบ เพื่อเรียนรู้วิธีการเรียกใช้งาน Web Services

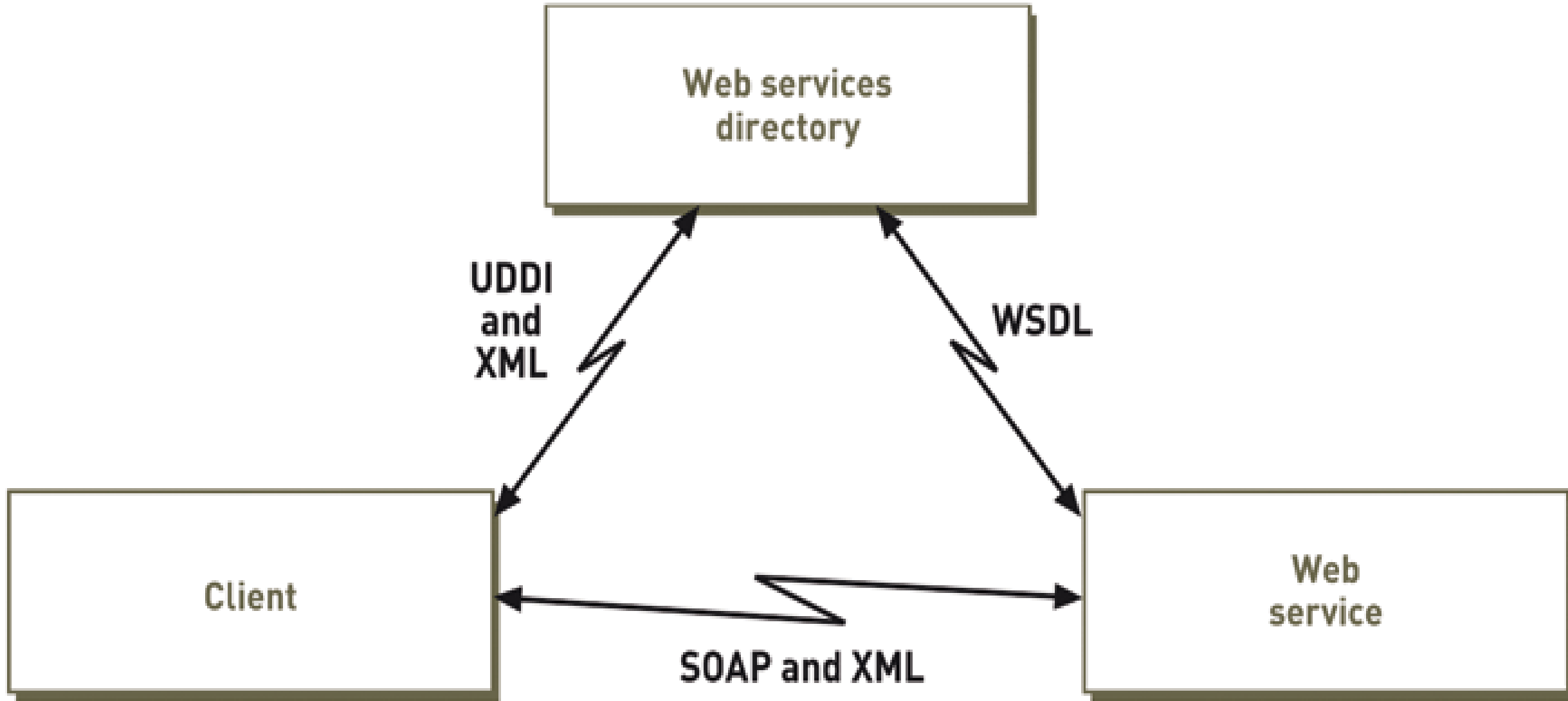
▶ 4. UDDI (Universal Description, Discovery, and Integration) เป็นระบบมาตรฐานในการอธิบายและค้นหา Web Services โดยเป็นตัวกลางให้ provider มาลงทะเบียนไว้ โดยใช้ไฟล์ WSDL บอกรายละเอียดของบริษัทและบริการที่มีให้ ทำให้ Requestor สามารถค้นหาและทราบว่าบริษัทมีผลิตภัณฑ์และบริการอะไรบ้าง สามารถติดต่อขอดำเนินธุรกิจการค้ากับบริษัทได้โดยอัตโนมัติผ่านทาง Web Services

WEB SERVICES, SERVICE ARCHITECTURES (ต่อ)



ที่มา <http://th.wikipedia.org>

Web Services Architecture (continued)



DESIGN PHASE : CONTROL AND SECURITY

1) Design ส่วนของ CONTROL

-เพื่อป้องกันระบบ และข้อมูล, Database ภายในระบบ เช่น

: ข้อมูลมีความปลอดภัย ให้เฉพาะ User ที่มี authorize (Authorize User) และ Data Permission เท่านั้น

: มีการตรวจสอบ (verify) ความถูกต้องของข้อมูลก่อนบันทึก

: มีการป้องกันจาก Hacker

: มีระบบ Authentication ที่ดี

ฯลฯ

-การควบคุมระบบ มี ส่วนควบคุม 5 ส่วนควบคุมได้แก่

: Operating System

: Database

: Input/Output

: Application Program

: Network Access

2) การออกแบบการควบคุม ตามระดับของ CONTROL

2.1) การออกแบบระดับ Integrity Controls เป็นการควบคุมระดับภายใน ในส่วนโปรแกรมประยุกต์ ที่ควบคุมให้ User สามารถใช้ระบบได้อย่างถูกต้อง ได้แก่

- ข้อมูลสั่งซื้อสินค้าที่ลูกค้าบันทึกและประมวลผลได้อย่างถูกต้อง
- ข้อมูลที่ผู้จัดการปรับปรุงสถานะ การชำระเงินใบสั่งซื้อที่มีการชำระเงินแล้ว โปรแกรมต้องปรับปรุงข้อมูลได้ถูกต้อง
- โปรแกรมสามารถคำนวณรายการขายย่อย และรวมราคาขายทั้งหมดรวม VAT ได้อย่างถูกต้องแม่นยำ

2.1.1) การควบคุมส่วน INPUT INTEGRITY CONTROLS โดย APPLICATION PROGRAM เพื่อให้ผู้ใช้ สามารถบันทึกข้อมูลเข้าสู่ระบบได้อย่างถูกต้องแม่นยำ ได้แก่

- Completeness Control
- Data Validation Control

- **Value Limit Control**
- **Field Combination Control**

2.1.2) การควบคุมส่วน DATABASE INTEGRITY CONTROLS

- **ACCESS CONTROL**
- **DATA ENCRYPTION** ใส่อัฒข้อมูลใน database และระหว่างการรับ ส่งข้อมูล
- **TRANSACTION CONTROL** ควบคุมการดำเนินการกับข้อมูลเพื่อความปลอดภัยในการประมวลผล เช่น Username ใคร , process อะไร กับข้อมูล..... วัน เวลา port.....
- **UPDATE CONTROL** โดยคำนึงถึง Concurrent ที่ต้องการแก้ไขพร้อมกัน ให้ username ใดก่อน
- **Backup and Recovery Control**

2.1.3) การควบคุมส่วน OUTPUT INTEGRITY CONTROLS เพื่อให้ output ที่ได้จากการประมวลผลในระบบ ได้สารสนเทศที่ถูกต้อง แม่นยำ และถูกส่งไปยังปลายทางที่ถูกต้อง (**DESTINATION CONTROL**)

2.2) การออกแบบระดับ Security Controls เป็นการควบคุมระดับภายนอก ในส่วนติดต่อ

ภายนอกระบบไม่ให้ได้ถูกบุกรุก หรือ ถูกโจมตี หรือ ถูกทำลาย หรือ ถูกคุกคาม

โดยปกติ จะการควบคุมส่วน INTERFACE ระหว่างระดับภายในกับระดับภายนอกมักเป็นหน้าที่

ของ OPERATING SYSTEM , NETWORK , NETWORK OS , DBMS

-เป้าหมายของการออกแบบระดับ SECURITY CONTROL เพื่อ

: ควบคุมความปลอดภัย เช่น การรับส่งข้อมูลระหว่าง network,

: ปกป้องการโจมตีจากภายนอกระบบ เช่น HACKER, VIRUS

: ติดตั้ง FIREWALL

: VERIFY USERNAME, PASSWORD, AUTHORIZE, PERMISSION,